Object Oriented Programming [FECEg-3142]

CHAPTER TWO:

Introduction to Java elements

Outline

- Introduction to the Java development environment
- Variables and their declaration
- Introduction to classes and objects
- input/output
- Conditional statements and loops
- Arrays

Introduction to the Java development environment

- **Java** was developed by a team led by **Jame Gosling** at **Sun Microsystems**.
- Originally called **Oak**, it was designed in 1991 for use in embedded chips in consumer electronic appliances.
- In 1995, renamed **Java**, it was redesigned for developing **Internet applications**.
- As stated by Sun, Java is simple, object oriented, distributed, interpreted, robust, secure, architecture neutral, portable, high performance, multithreaded, and dynamic.

The Java Language Specification, API, JDK, and IDE

- The **Java language specification** and **Java API** define the **Java standard**.
- The **Java language specification** is a technical definition of the language that includes the **syntax** and **semantics** of the Java programming language.
- The **application program interface** (API) contains predefined **classes** and **interfaces** for developing Java programs.
- The Java language specification is stable, but the API is still expanding.

Cont'd...

- Java is a full-fledged and powerful language that can be used in many ways.
- It comes in three editions:
  - Java Standard Edition (Java SE),
  - Java Enterprise Edition (Java EE), and
  - Java Micro Edition (Java ME).
- **JDK** consists of a set of separate programs, each invoked from a command line, for developing and testing Java programs.
- NetBeans(IDE)

Creating, Compiling, and Executing a Java Program

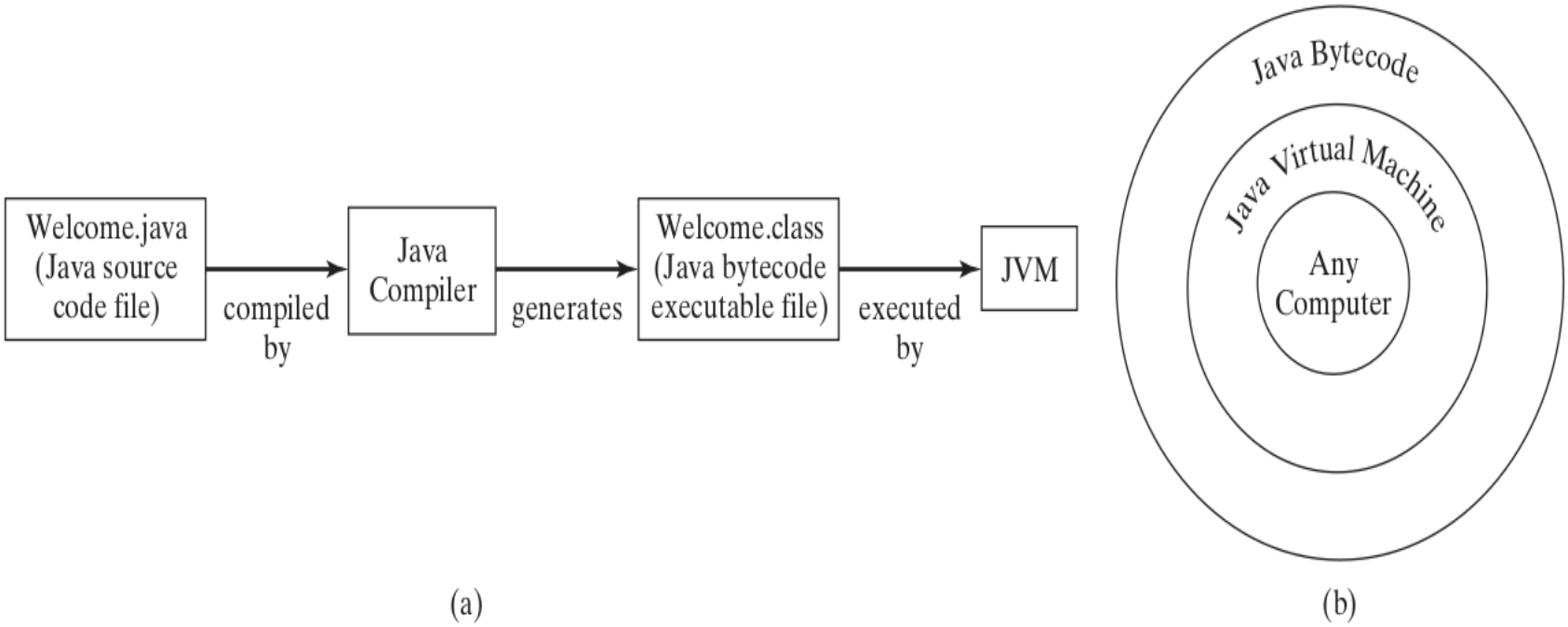


Figure (a) Java source code is translated into bytecode. (b) Java bytecode can be executed on any computer with a Java Virtual Machine.

A Simple Java Program

- Let us begin with a simple Java program that displays the message "**Welcome to Java!**" on the console.
- **Console** refers to text entry and display device of a computer.

```
1. public class Welcome {
2. public static void main(String[] args) {
3. // Display message Welcome to Java! to the console
4. System.out.println("Welcome to Java!");
5. }
6. }
```

Cont'd...

- Line 1 defines a **class**.
  - Every Java program must have at least one class.
  - Each **class** has a **name**.
  - By convention, **class names start** with an **uppercase letter**.

Cont'd...

- Line 2 defines the **main method**.
  - In order to run a **class**, the class must contain a **method** named **main** .
  - The program is executed from the main method.
  - A **method** is a construct that contains statements.
  - The main method in this program contains the System.out.println statement.
  - This statement prints a message " Welcome to Java! " to the console (line 4).
- Every **statement** in Java **ends** with a **semicolon** (**;**), known as the **statement terminator**.

Cont'd...

- Line 3 is a **comment** that documents what the program is and how it is constructed.
  - **Comments** help programmers to **communicate** and **understand** the program.
  - They are not programming statements and thus are **ignored** by the **compiler**.

Cont'd...

- In Java, comments are preceded by **two slashes** (//) on a line, called a **line comment**, or enclosed between /* and */ on one or several lines, called a block comment.
- When the compiler sees // , it ignores all text after //on the same line.
- When it sees /* , it scans for the next */ and ignores any text between /* and */ .

Cont'd...

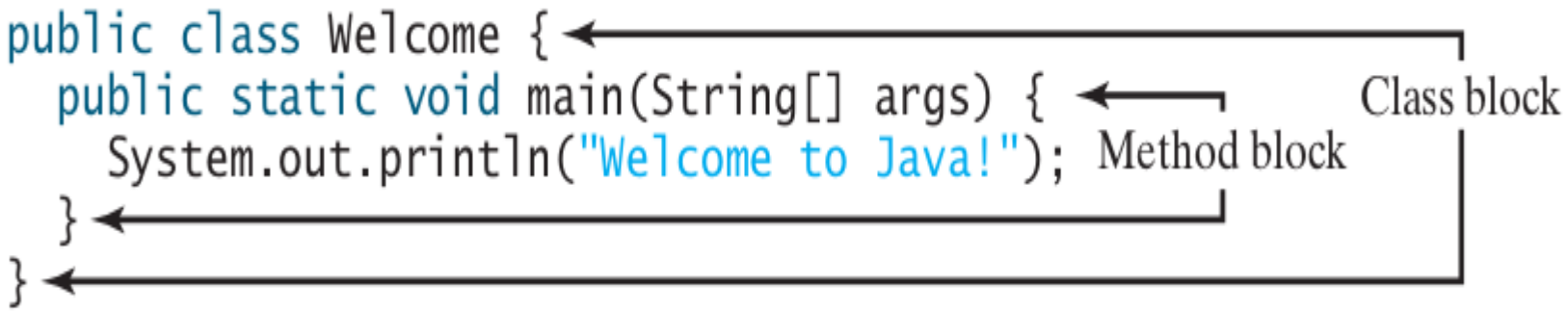
- Here are examples of comments:

```
// This application program prints Welcome to Java!
/* This application program prints
   Welcome to Java! */
/* This application programprints
   Welcome to Java! */
```

Cont'd...

- A **pair of braces** in a program forms a **block** that **groups** the program's components.
- In Java, each block begins with an opening brace ({) and ends with a closing brace (}).
- Every **class** has a class block that groups the **data** and **methods** of the class.
- Every **method** has a method block that groups the **statements in the method**.
- Blocks can be nested, meaning that one block can be placed within another.

Example


public class Welcome {
public static void main(String[] args) {
System.out.println("Welcome to Java!");
}
}
Class block
Method block

Identifiers, operators and Expressions

Variable and primitive data type

Identifiers

- The names of things that appear in the program.
- All identifiers must obey the following rules:
  - An identifier is a sequence of characters that consists of **letters**, **digits**, **underscores (_)**, **and dollar signs ($)**.
  - An identifier must start with a **letter**, an **underscore** (_), or **a dollar sign** (**$**).
  - It cannot start with a digit.
  - An identifier cannot be a reserved word.
  - An identifier cannot be true , false , or null .
  - An identifier can be of any length.

Variables and their declaration

- **Variables** are used to store values to be used later in a program. They are called **variables** because their values can be changed.
- Variables are for representing data of a certain type.
- To use a variable, you declare it by telling the compiler its name as well as what type of data it can store.
- The variable declaration tells the compiler to allocate appropriate memory space for the variable based on its data type.

Cont'd...

- The syntax for declaring a variable is

  datatype variableName;

  Here are some examples of variable declarations:

```
int count;
// Declare count to be an integer variable;
double radius;
// Declare radius to be a double variable;
double interestRate;
// Declare interestRate to be a double variable;
```

Cont'd...

- If variables are of the same type, they can be declared together, as follows:

  datatype variable1, variable2, ..., variablen;

- The variables are separated by commas.

  For example,

```
int i, j, k;
// Declare i, j, and k as int variables
```

Cont'd...

- Variable initialization.

```
int count = 1;
```

This is equivalent to the next two statements:

```
int count;
count = 1;
```

Data type

- Every variable in Java has a **data type.**
- **Data types** specify, the **size** and **type** of values that can be stored
- In java there are two categories of data types:
  - *I.* ***Primitive (Basic/Simple) types***: consists of int, float, char and Boolean.
  - *II.* ***Reference types***: consists of interfaces, classes, and arrays.

Integer

- Java has four integer data types:

Data Type	Range	Storage Size
Byte	-128 to 127	1 byte
Short	- 32,768 to 32,767	2 byte
Int	-2,147,483,648 to 2,147,483,647	4 byte
Long	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807	8 byte

Floating Point Numbers

- Is any number with a decimal point.
- Java has two floating point data types.
- use the double type, because it is more accurate than the float type.

Data Type	Range	Storage Size
Float	Negative range: -3.4028235E + 38 to -1.4E – 45 Positive range: 1.4E–45 to 3.4028235E + 38	4 byte
Double	Negative range: -1.7976931348623157E + 308 to -4.9E–324 Positive range: 4.9E - 324 to 1.7976931348623157E + 308	8 byte

Character Data type

- The character data type, char , is used to represent a single character.
- A character literal is enclosed in single quotation marks.

  Example

  ```
  char letter = 'A';
  char numChar = '4';
  ```

Boolean data type

- Are represented as being either **true** or **false**
- Variables declared from the **Boolean data type** can accept only one of two values: **true** or **false.**

Example

```
boolean lightsOn = true;
```

The String Type

- The **char type** represents only **one character**.
- To represent a **string of characters**, use the data type called **String**.

  Example

  ```
  String message = "Welcome to Java";
  ```

Type casting

- **Casting** is an operation that converts a value of **one data type** into a value of **another data type**.
- The syntax is the **target type** in **parentheses**, followed by the **variable's name** or **the value to be cast**.

Example

```
double var=3.4;
int var_2= (int)var;
```

Operators and Expressions

- is a symbol that tells the computer to perform certain mathematical or Logical manipulations are used in programs to manipulate data and variables.
- Java operators can be classified into a number of related categories as below:
    1. Arithmetic operators
    2. Relational operators
    3. Logical operators
    4. Assignment operators
    3. Increment and decrement operators
    6. Conditional operators

Arithmetic operators

- are used to construct mathematical expressions as in algebra

Operators	Meaning
+	Addition or unary plus
-	Subtraction or unary minus
*	Multiplication
/	Division
%	Modulo division (Remainder)

Relational Operators

- Use to compare two values.

Operator	Meaning
<	Less than
<=	Less than or equal to
>	Greater than
>=	Greater than or equal to
==	Equal to
!=	Not equal to

Logical operator

- Logical operators, also known as Boolean operators, operate on Boolean values to create a new Boolean value.

Operator	Meaning
&&	Logical AND
||	Logical OR
!	Logical NOT

Assignment operator

- Used to assign the value of an expression to a variable
- usual assignment operator is =

  Y=7+9;

  Y=x+z;
- Java has short hand assignment operators like +=, -=, *=

  Y+=7; means similar to y=y+7;

Increment(++) and decrement(--) operator

- The operator ++ add 1 to the operand while subtract 1.
- ++ m or m++ is equivalent to m+1
- ++m and m++ are different when they used in expression

  m=5;
- y=++m; this make both y and m equal to 6
- y=m++; this make m equals 6 but y will be 5

Conditional Expressions

- The syntax is shown below:

  boolean-expression ? expression1 : expression2;

- The result of this **conditional expression** is **expression1** if boolean-expression is **true**; otherwise the result is **expression2** .

- Suppose you want to assign the larger number between variable num1 and num2 to max .

- You can simply write a statement using the conditional expression:

```
max = (num1 > num2) ? num1 : num2;
```

input/output

- Java uses **System.out** to refer to the standard output device and **System.in** to the standard input device.
- By default the output device is the display monitor, and the input device is the keyboard.
- To perform console output, you simply use the **println** method to display a **primitive value** or a **string** to the console.

Cont'd...

- Console input is not directly supported in Java, but you can use the **Scanner class** to create an object to read input from **System.in** , as follows:

  Scanner input = **new Scanner**(**System.in**);

Cont'd...

TABLE Methods for Scanner Objects

Method	*Description*
`nextByte()`	reads an integer of the `byte` type.
`nextShort()`	reads an integer of the `short` type.
`nextInt()`	reads an integer of the `int` type.
`nextLong()`	reads an integer of the `long` type.
`nextFloat()`	reads a number of the `float` type.
`nextDouble()`	reads a number of the `double` type.
`next()`	reads a string that ends before a whitespace character.
`nextLine()`	reads a line of text (i.e., a string ending with the *Enter* key pressed).

Cont'd...

```
import java.util.Scanner;
public class ComputeAreaWithConsoleInput {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        // Create a Scanner object
        // Prompt the user to enter a radius
        System.out.print("Enter a number for radius: ");
        double radius = input.nextDouble() ;
        double area = radius * radius * 3.14159; // Compute area
        // Display result
        System.out.println("The area for the circle of radius " +
        radius + " is " + area);
    }
}
```

Conditional statements and loops

- Like all high-level programming languages, Java provides **selection statements** that let you choose actions with two or more alternative courses.
- Selection statements use **conditions**.
- **Conditions** are **Boolean expressions**.

Cont'd...

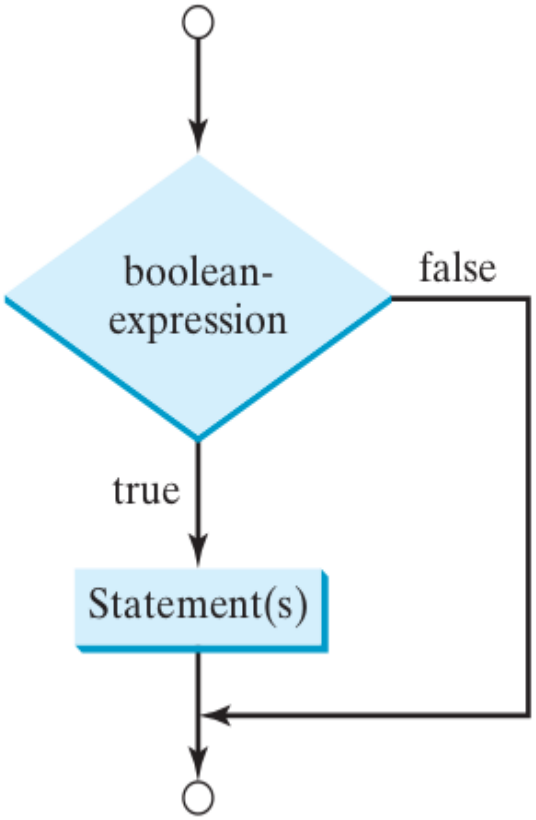
- Java has several types of selection statements:
    1. The if statements
    2. The if-else statements
    3. Nested if statements
    4. Switch statements

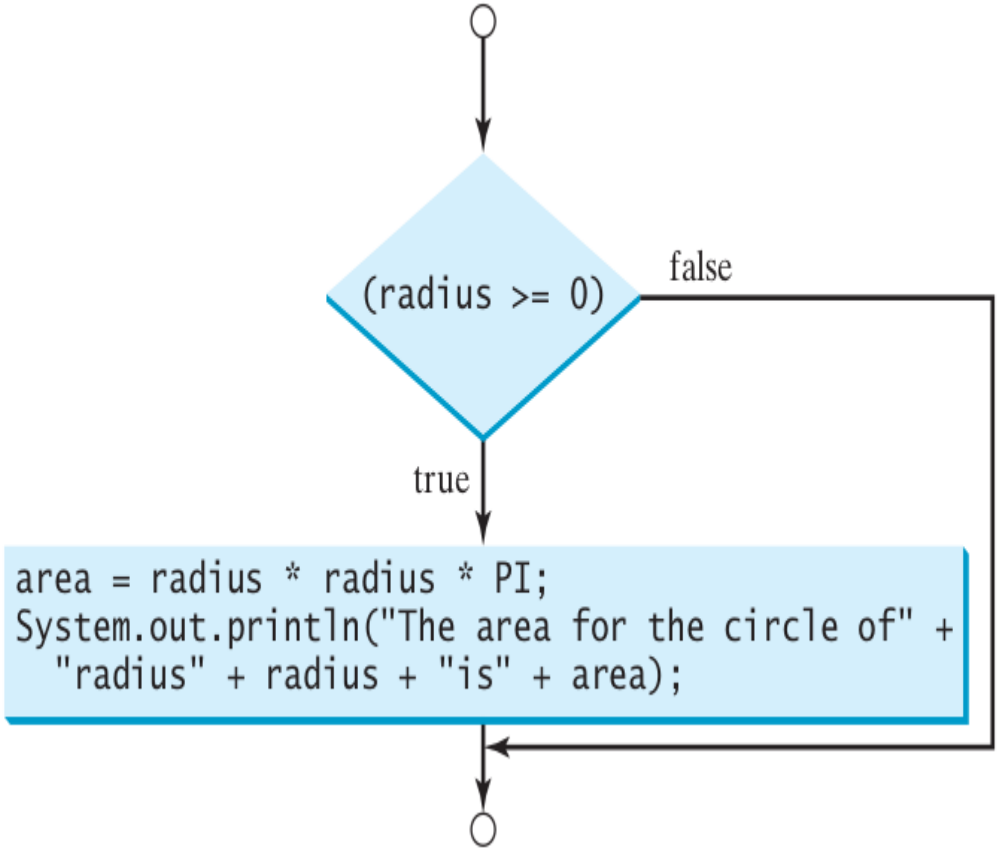
1. The if statement

- **if statement** executes an action if and only if the **condition** is **true** .
- The syntax for a if statement is shown below:

```
if (boolean-expression) {
statement(s);
}
```

Cont'd...





An **if statement** executes statements if the **boolean-expression** evaluates to **true** .

Cont'd...

```
import java.util.Scanner;
public class SimpleIfDemo {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        System.out.println("Enter an integer: ");
        int number = input.nextInt();
        if (number % 5 == 0)
            System.out.println("HiFive");
        if (number % 2 == 0)
            System.out.println("HiEven");
    }
}
```

2. The if-else statements

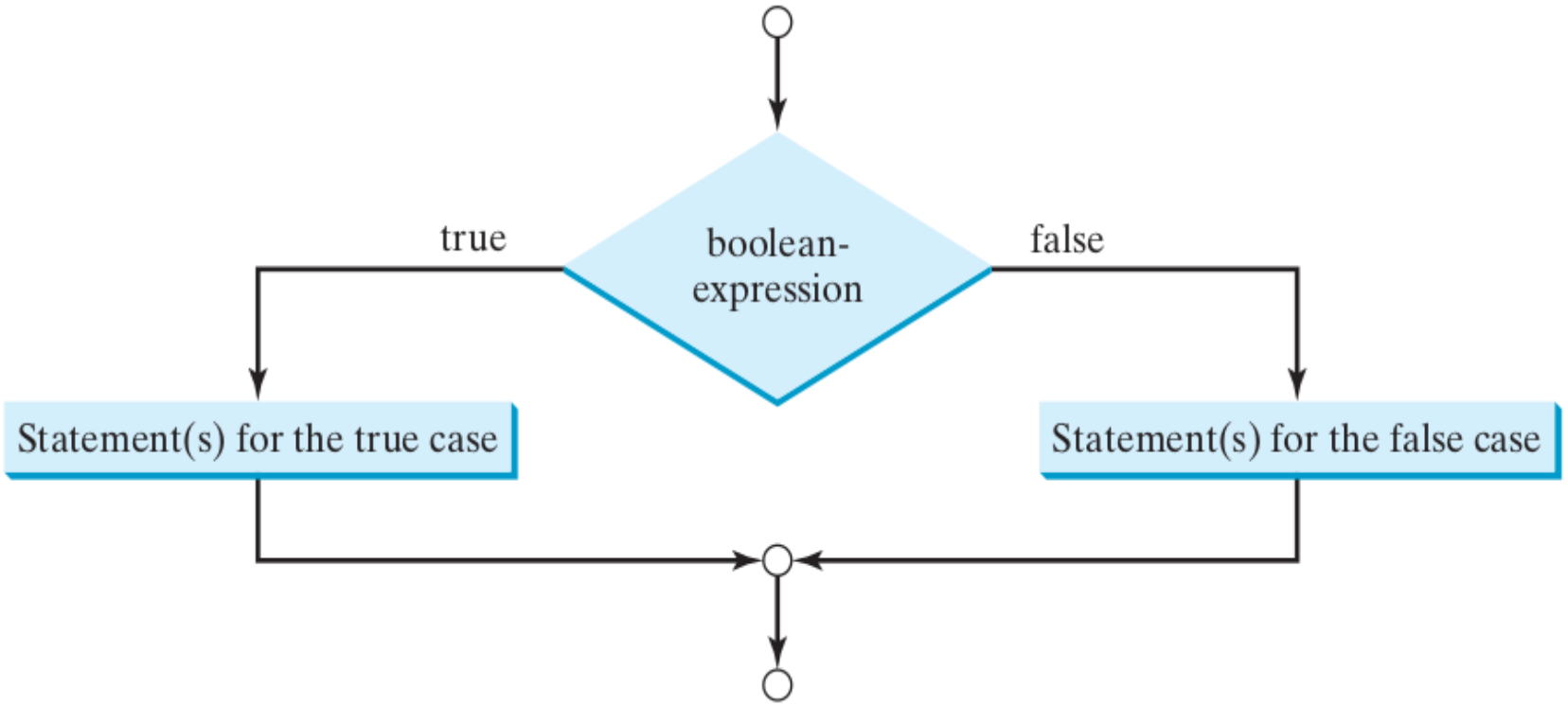
- The **if statement** takes an action if the specified **condition** is **true** . If the **condition** is **false** , **nothing is done**.
- But what if you want to take alternative actions when the condition is false ? You can use a if -else statement.
- The actions that a if-else statement specifies differ based on whether the condition is true or false .

Cont'd...

- Here is the syntax for a if-else statement:

```
if (boolean-expression) {
    statement(s)-for-the-true-case;
}
else {
    statement(s)-for-the-false-case;
}
```

Cont'd...



An **if - else** statement executes statements for the **true** case if the **boolean-expression** evaluates to **true** ; otherwise, statements for the false case are executed.

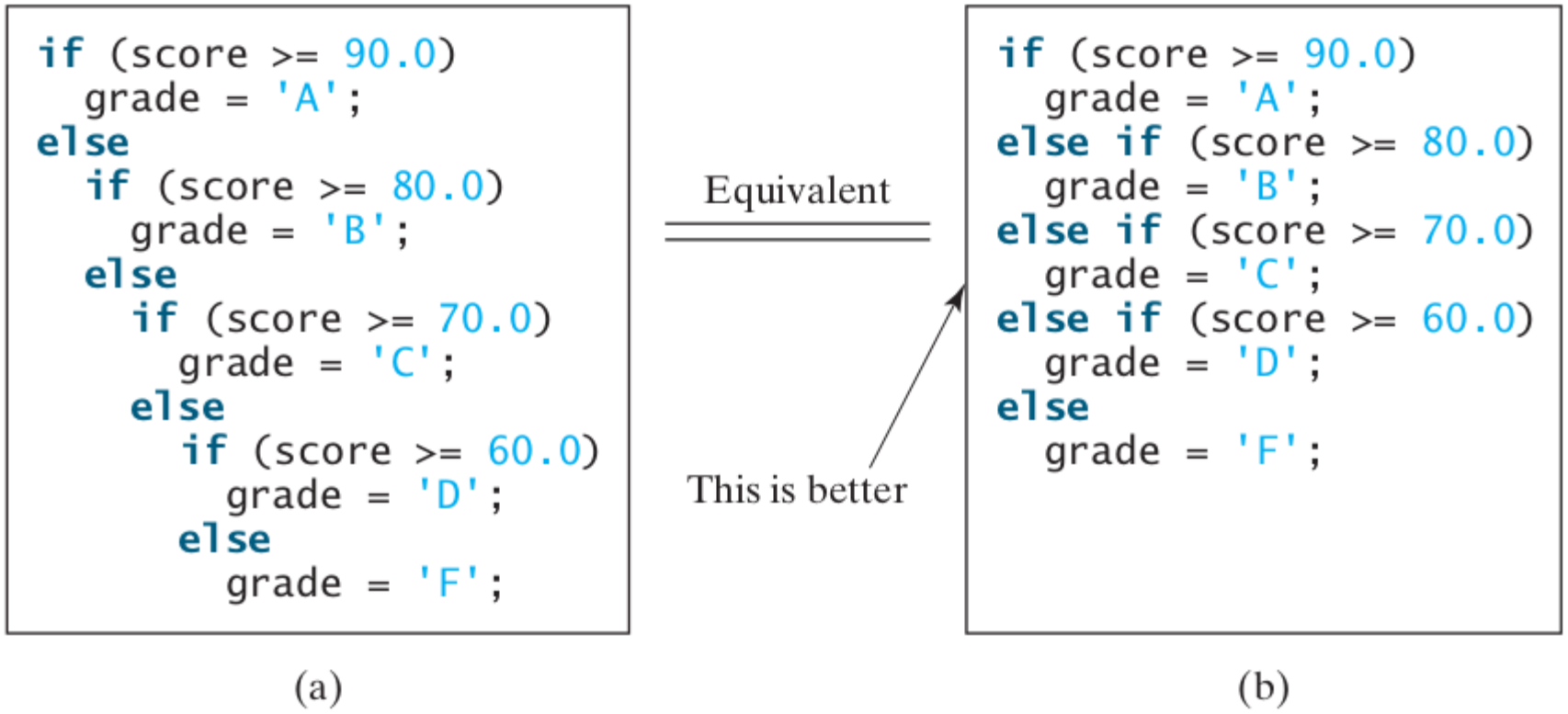
Cont'd...

```
if (radius >= 0) {
    area = radius * radius * PI;
    System.out.println("The area for the circle
    of radius " + radius + " is " + area);
}
else {
    System.out.println("Negative input");
}
```

3. Nested if Statements

- The statement in an if or if ... else statement can be any legal Java statement, including another if or if ... else statement.
- The inner if statement is said to be nested inside the outer if statement.
- The inner if statement can contain another if statement; in fact, there is no limit to the depth of the nesting.
- The nested if statement can be used to implement multiple alternatives.

Cont'd...



A preferred format for multiple alternative if statements is shown in (b).

4. Switch Statements

- Java provides a **switch statement** to handle **multiple conditions** efficiently.

Cont'd...

- Here is the full syntax for the switch statement:

```
switch (switch-expression) {
        case value1:
           statement(s)1;
           break;
        case value2:
           statement(s)2;
           break;
           ...
        case valueN:
           statement(s)N;
           break;
        default:
           statement(s)-for-default;
}
```

Cont'd...

The switch statement observes the following rules:

- The **switch-expression** must yield a value of **char** , **byte** , **short** , **or** **int type** and must always be enclosed in **parentheses**.
- The **value1** , ... , and **valueN** must have the **same data type** as the value of the **switch-expression** . Note that **value1** , ... , and **valueN** are **constant expressions**, meaning that they cannot contain **variables**, such as 1 + x .
- When the value in a **case statement** matches the value of the **switch-expression**, the statements starting from this case are executed until either a **break statement** or the **end of the switch statement** is reached.

Cont'd...

- The keyword **break** is optional. The break statement immediately **ends the switch statement**.
- The **default case**, which is optional, can be used to perform actions when none of the specified cases matches the switch-expression .
- The **case statements** are checked in **sequential order**, but the order of the cases (including the default case) does not matter.
- However, it is good programming style to follow the logical sequence of the cases and place the default case at the end.

The break and continue Statements

- **Break**
  - Causes immediate **exit** from a **while**, **for**, **do/while** or **switch** structure
  - Common uses of the **break** statement:
    - Escape early from a loop
    - Skip the remainder of a **switch** structure

Cont'd...

- **Continue**
  - Skips the remaining statements in the body of a **while**, **for** or **do/while** statements and proceeds with the next iteration of the loop.
  - In **while** and **do/while**, the loop-continuation test is evaluated immediately after the **continue** statement is executed.
  - In the **for** statements, the increment expression is executed, then the loop-continuation test is evaluated.

Cont'd...

```
public class BreakContinue {
        public static void main(String[] args){
          int i=0;
           while(i<=10){
              i++;
              if(i==4){
                  continue;
              }
              if(i==8){
              break;
              }
               System.out.println(i);
          }
        }
}
```

Loops

- **Loops** are constructs that control repeated executions of a block of statements.
- Java provides **three types** of loop statements:

  1.The while loops

  2.The do - while loops and

  3. The for loops

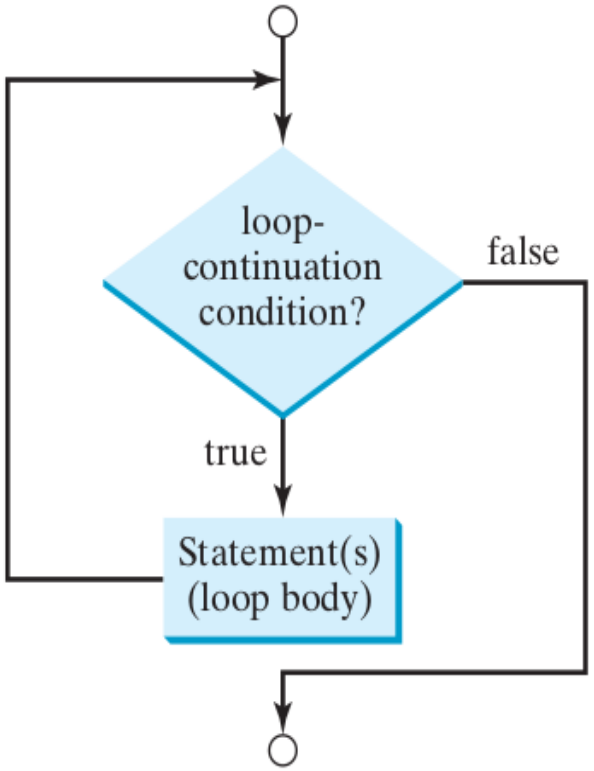
1.The while Loop

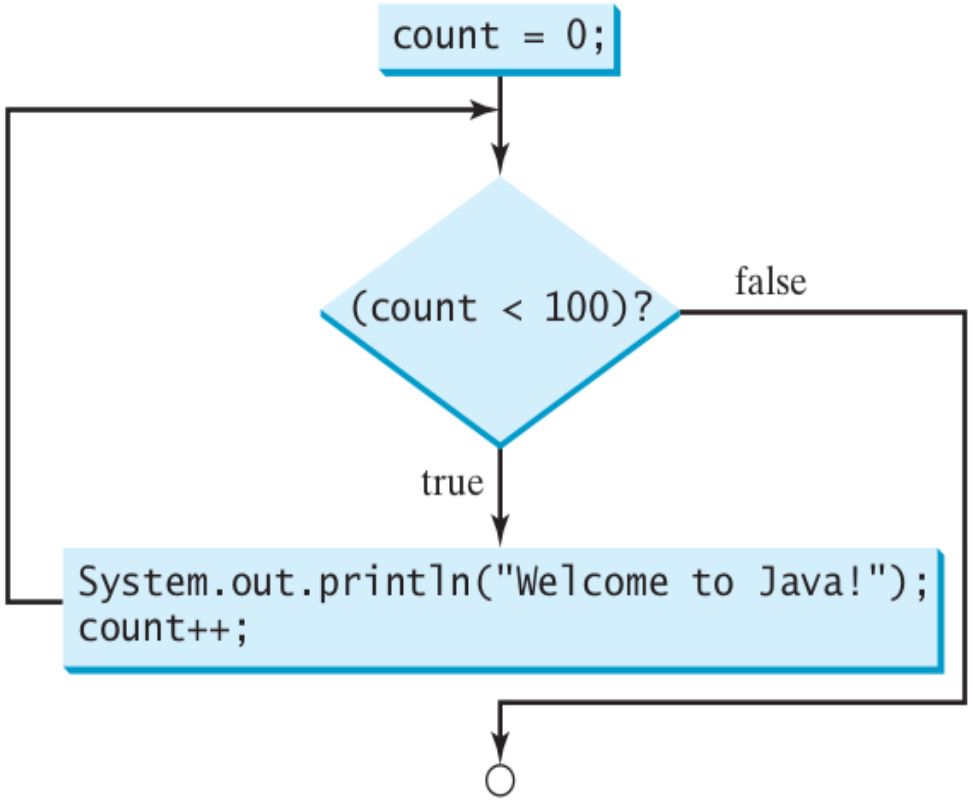
- The syntax for the while loop is as follows:

```
while (loop-continuation-condition) {
// Loop body
Statement(s);
}
```

- The part of the loop that contains the statements to be **repeated** is called the **loop body**.
- A **one-time execution** of a **loop body** is referred to as an **iteration** of the loop.

Cont'd...





The while loop repeatedly executes the statements in the loop body when the loop-continuation-condition evaluates to true .

Cont'd...

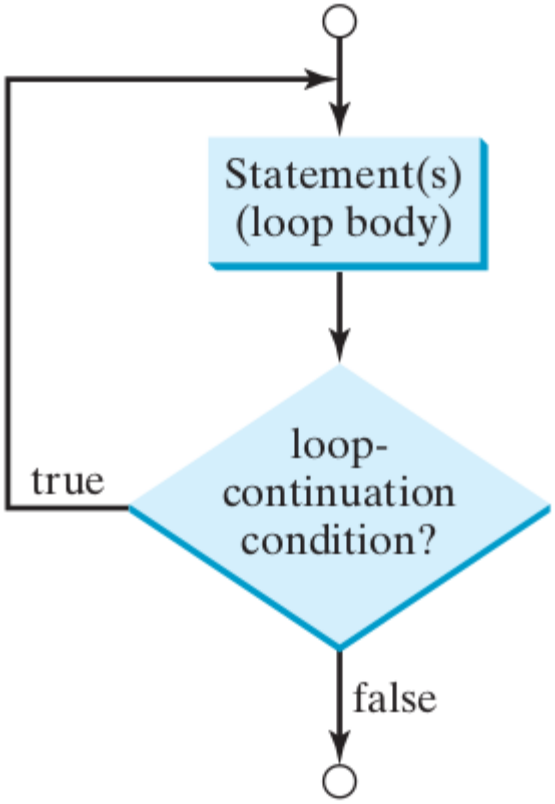
- Each **loop** contains a **loop-continuation condition**, a **Boolean expression** that **controls the execution** of the **body**.
- It is evaluated each time to determine if the loop body is executed.
- If its evaluation is **true** , the **loop body** is executed; if its evaluation is **false** , the entire **loop terminates** and the program control turns to the statement that follows the while loop.

2. The do-while Loop

- The do - while loop is a variation of the while loop.
- Its syntax is given below:

```
do {
// Loop body;
Statement(s);
} while (loop-continuation-condition);
```

Cont'd...



The **do-while loop** executes the **loop body first**, then checks the **loop-continuation-condition** to determine whether to **continue** or **terminate** the loop.

Cont'd...

- The loop body is executed first.
- Then the loop-continuation-condition is evaluated. If the evaluation is true , the loop body is executed again; if it is false , the do – while loop terminates.
- The difference between a while loop and a do - while loop is the order in which the loop-continuation-condition is evaluated and the loop body executed.
- The while loop and the do - while loop have equal expressive power. Sometimes one is a more convenient choice than the other.

Cont'd...

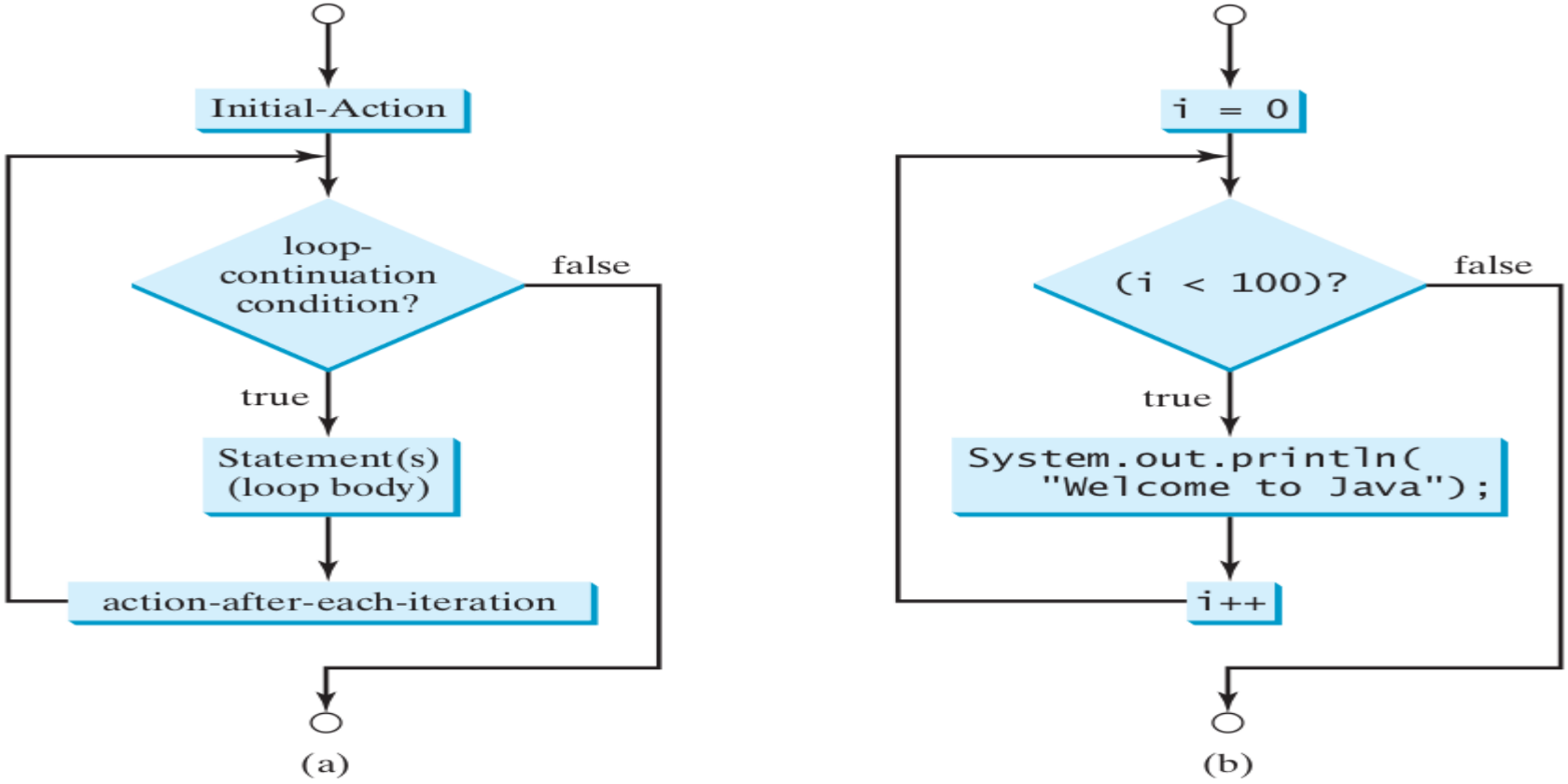
```
import java.util.Scanner;
public class TestDoWhile {
/* Main method */
   public static void main(String[] args) {
      int data;
      int sum = 0;
      // Create a Scanner
      Scanner input = new Scanner(System.in);
      // Keep reading data until the input is 0
      do {
         // Read the next data
         System.out.print("Enter an int value (the program exits if the input is 0): ");
         data = input.nextInt();
         sum += data;
         } while (data != 0);
      System.out.println("The sum is " + sum);
   }
}
```

3. The for Loop

- The syntax of a for loop is as shown below:

```
for (initial-action; loop-continuation-condition;action-after-each-iteration) {
// Loop body;
Statement(s);
}
```

Cont'd...



- A **for loop** performs an **initial action once**, then repeatedly executes the statements in the **loop body**, and performs an action after an iteration when the **loop-continuation-condition** evaluates to **true** .

Cont'd...

- The for loop statement starts with the keyword **for** , followed by a pair of **parentheses** enclosing the control structure of the loop.
- This structure consists of **initial-action** , **loop-continuation-condition** , and **action-after-each-iteration** .
- The control structure is followed by the **loop body** enclosed **inside braces**.
- The initial-action , loop- continuation-condition , and action-after-each-iteration are separated by **semicolons**.

Cont'd...

- A for loop generally uses a variable to control how many times the loop body is executed and when the loop terminates.
- This variable is referred to as a control variable.
- The initial-action often initializes a control variable, the action-after-each-iteration usually increments or decrements the control variable, and the loop-continuation-condition tests whether the control variable has reached a termination value.

Arrays

- Allows the creation of a group of variables with the same data type
- Consist of elements:
  - Each element behaves like a variable
- Can be:
  - One dimensional
  - Multi-dimensional

Declaring and Accessing Arrays

- To use an array in a program, you must **declare a variable** to **reference the array** and specify the array's **element type**.
- Here is the syntax for declaring an array variable:

  elementType[] arrayRefVar;
- The **elementType** can be any **data type**, and all elements in the array will have the same data type.

  Example

  double[] myList;

Cont'd...

- Unlike declarations for primitive data type variables, the declaration of an array variable does not allocate any space in memory for the array.
- It creates only a storage location for the reference to an array.
- If a variable does not contain a reference to an array, the value of the variable is **null** .
- After an array variable is declared, you can create an array by using the **new** operator with the following syntax:

arrayRefVar = **new** **elementType[arraySize];**

Cont'd...

- This statement does two things:
  - It creates an array using new elementType[arraySize] ;
  - It assigns the reference of the newly created array to the variable arrayRefVar.
- Declaring an **array variable**, **creating an array**, and assigning the **reference of the array** to the **variable** can be combined in one statement, as shown below:

elementType arrayRefVar = **new** elementType[arraySize];

or

elementType arrayRefVar[] = new elementType[arraySize];

Cont'd...

- Here is an example of such a statement:

  double[] myList = new **double**[10];

- To assign values to the elements, use the syntax:

  arrayRefVar[index] = value;

- The array elements are accessed through the index. Array indices are 0 based; that is, they range from **0** to **arrayRefVar.length-1** .

- Syntax

  arrayRefVar[**index**];

End